Product Requirements Document (PRD)

TriagePH: AI-Powered Health Intelligence & Triage System

1. 📌 Product Overview

Product Name: TriagePH **Concept:** An AI-driven health intelligence and triage system.

Problem Statement: Public healthcare systems in the Philippines are often overloaded. Patients frequently misjudge the severity of their symptoms, leading to delayed treatment for critical conditions or unnecessary ER visits for minor ailments. There is a significant lack of an accessible, data-driven tool that can:

- Help users accurately assess symptom urgency.
- Learn from aggregated health data patterns.
- Provide population-level health insights and early warnings.

Solution: A data-driven AI triage web platform designed to:

- Classify symptom severity in real-time.
- Learn from historical and user-generated data.
- Provide comprehensive health trend analytics and predictive insights.

2. 🎯 Goals & Objectives

Primary Goals:

- **Instant Triage:** Provide fast, reliable triage classification (low / medium / high urgency).
- **Optimize Resource Allocation:** Reduce unnecessary ER visits by guiding users to the appropriate level of care.
- **Health Intelligence:** Build a dashboard for localized health trend analysis and tracking.

Secondary Goals:

- **Data Collection:** Accumulate anonymized symptom data for public health insights.
- **Trend Detection:** Detect emerging health trends and localized outbreaks (e.g., spikes in dengue-like symptoms).
- **Technical Demonstration:** Showcase a seamless integration of AI and data systems for hackathon evaluation and future public health adoption.

3. 👥 Target Users

- **Patients / General Public:** Individuals and families, especially those without immediate or easy access to doctors, seeking initial symptom assessment.
- **Barangay Health Workers (BHWs):** Local health officials using the tool to triage community members or monitor local health trends.
- **Hackathon Judges:** The primary evaluation audience focusing on innovation, technical implementation, and social impact.

4. ⚙️ Core Features

🧠 4.1 AI Triage Engine (Core Feature)

- **Input Mechanisms:** Accepts natural language descriptions of symptoms (Text in Tagalog, English, or Taglish). Optional inputs include age, symptom duration, and body temperature.
- **Output & Classification:**
    - 🟢 **Low Urgency:** Self-care or pharmacy visit recommended.

    - 🟡 **Medium Urgency:** Clinic visit or teleconsultation recommended.
    - 🔴 **High Urgency:** Immediate ER visit recommended.
- **Additional Outputs:** Potential condition category (explicitly stating it is not a medical diagnosis) and clear recommended actions.

📊 4.2 Health Data Engine

- **Functionality:** Stores anonymized symptom reports securely and aggregates data for macro-analysis.
- **Stored Data Points:** Symptoms, severity classification, timestamps, and optional location data (Barangay/City level).

📈 4.3 Intelligence Dashboard

- **Analytics Visualization:** A web-based dashboard displaying:
    - 📊 Most common symptoms reported daily/weekly.
    - 📈 Trend spikes (e.g., unusual increases in fever or respiratory cases).
    - 🏥 Percentage of cases flagged for ER-level urgency.
    - 🌡️ Correlations with external factors (e.g., heat waves, seasonal changes).

🔄 4.5 Feedback Loop System

- **User Validation:** After receiving a recommendation, the system follows up (or asks before exit): *"Did you seek medical care?" (Yes/No)*.
- **Continuous Improvement:** The system uses this outcome data to refine and adjust future scoring algorithms.

📑 4.6 Explainability Module

- **Transparency:** For every classification, the system provides a brief rationale.
- **Trigger Identifiers:** Highlights the specific key symptoms (e.g., "shortness of breath") that drove the urgency score, ensuring user trust.

5. 🧠 System Design & Architecture

Frontend:

- **Tech Stack:** React.js + Tailwind CSS.
- **UI/UX & Branding:** Implementing a major UI redesign alongside a new logo to enhance usability and establish a modern brand identity.
- **Key Interfaces:**
    - **Patient Interface (Public):**
        - *Input Page:* The symptom checker interface for the general public.
        - *Result Page:* Displays the AI triage output, explanations, and recommended actions.
    - **Admin Interface (Healthcare Facility):**
        - *Dashboard Page:* The analytics and intelligence visualization center for admins and health workers to monitor population health trends and triage aggregate data.

Backend:

- **Tech Stack:** Node.js or Flask API.
- **Responsibilities:** Routing symptom text to the classification engine, handling data storage, and processing analytics queries.

AI / Logic Layer:

- *Option A (Hackathon MVP):* Rule-based scoring system matching keywords and flags.
- *Option B (Advanced):* ML Model (Logistic Regression / Decision Tree) trained on symptom datasets.
- *Option C (LLM Integration):* Fast LLM-based classification using the **Groq API** (migrating from Gemini for enhanced performance) with structured output generation.

Database:

- **Tech Stack:** PostgreSQL via **Neon** (migrating from Firebase for scalable relational data management).
- **Data Architecture:** Securely stores user inputs (strictly anonymized) and pre-computed aggregated statistics for fast dashboard rendering.

6. 📊 Data Flow

1. **Input:** User submits symptoms and optional metadata.
2. **Processing:** System sanitizes text and extracts key features (symptom tokens, duration).
3. **Classification:** AI model / Rule engine evaluates features and assigns an urgency score.
4. **Presentation:** Triage result and explanation are displayed to the user.
5. **Storage:** Anonymized data point is securely written to the database.
6. **Aggregation:** The Dashboard queries the database to update real-time insights.
7. **Refinement:** User feedback loop data is fed back into the dataset to improve future logic.

7. 📱 User Flow Example

- **Step 1:** User accesses the platform and navigates to "Check Symptoms".
- **Step 2:** User inputs text: *"lagnat, sakit ulo, at hilo"* (fever, headache, and dizziness).
- **Step 3:** The backend processes the input, identifying potential heat-related or dengue indicators.
- **Step 4:** Output is displayed:
  - **Urgency:** 🔴 High (or 🟡 Medium depending on duration/temperature).
  - **Context:** "Possible heat-related illness or viral infection."
  - **Recommendation:** "Drink fluids immediately. If symptoms persist or worsen, visit the nearest clinic or ER."